Software Requirements Specification

For

A Language independent tool for Document Creation
14-03-2026

Prepared by

Specialization	SAP ID	Name
Data Science	500124827	Kshitij Chandrakar
Data Science	500124609	Nitin Kumai
Data Science	500124417	Ansh negi

School Of Computer Science

UPES,

Dehradun- 248007, Uttarakhand

Revision History

Date	Change	Reason for Changes	Mentor Signature
14-03-2026	Initial SRS		

Table of Contents

1. Introduction
2. Project Description
3. System Requirements
4. Non-Functional Requirements
5. Other Requirements

- Appendix A: Glossary
- Appendix B: Analysis Models
- Appendix C: Issues List

1. INTRODUCTION

1.1 Purpose of the Project

The purpose of this project is to develop a unified, language-agnostic document creation and synchronization platform that overcomes the fragmented limitations of existing document authoring tools. The platform's core objectives are:

- **Develop a Language-Agnostic Platform:** To create a unified document creation system where the editing process is independent of any single markup or typesetting language. Users should be able to author, view, and manage documents regardless of the underlying format — be it LaTeX, Typst, Markdown, or DOCX.
- **Combine Strengths, Reduce Weaknesses:** To combine the best features of all existing methods (e.g., LaTeX's mathematical precision, Markdown's simplicity and portability, Typst's rapid compilation and scripting), while eliminating the individual drawbacks of each, such as LaTeX's steep learning curve, Markdown's limited layout support, and Word's instability with long documents.
- **Enable Synchronization:** To implement a robust synchronization engine that maintains consistency across different filetype representations of the same document, ensuring changes in one format are accurately reflected in all others. A master abstract document model will serve as the canonical source of truth across all format representations.

1.2 Target Beneficiary

The primary beneficiaries of this platform are:

- **Academic Researchers and Students** who must frequently switch between LaTeX (for journals), Markdown (for notes and documentation), and Word (for institutional submissions), and who lose time to format conversion and compatibility issues.
- **Technical Writers and Documentation Engineers** who manage documentation across multiple output formats (e.g., HTML, PDF, DOCX) and need a single-source authoring workflow.
- **Software Developers** who maintain project documentation in Markdown or reStructuredText but are required to submit deliverables in Word or PDF formats.
- **Publishers and Editors** who work across multi-format pipelines and require consistent, predictable output from a single document source.
- **Cross-disciplinary Teams** where members prefer or are constrained to different authoring environments, requiring interoperability without loss of fidelity.

1.3 Project Scope

This platform targets the domain of technical and academic document creation, with a focus on multi-format interoperability. The system will:

In Scope: - Parsing and representing documents from `.tex`, `.typ`, `.md`, and `.docx` source formats into a unified internal model. - Bidirectional conversion between supported formats using an

intermediary representation. - A synchronization engine that propagates edits made in one format representation to all others. - Offline-first operation with local-first data storage and optional sync. - Version control integration for tracking document history and resolving conflicts. - Multi-format export to PDF, HTML, DOCX, and plain text. - A unified editor interface with basic support for all source formats. - A filesystem management module for organizing document projects.

Out of Scope (Current Version): - Real-time collaborative editing (planned for a future release). - Mobile-native clients. - Full WYSIWYG editing for all formats simultaneously. - Cloud storage (offline-first; sync mechanisms are local or user-configured).

Deliverables: - A cross-platform desktop or web application with a unified document editor. - A core synchronization engine library. - A CLI tool for initiating format syncs and managing document state. - A filesystem and project management module. - Offline support with local conflict resolution. - Documentation and user guides.

1.4 References

- Typst Documentation: https://typst.app/docs/
- LaTeX Project Documentation: https://www.latex-project.org/help/documentation/
- Markdown Guide: https://www.markdownguide.org/
- Microsoft Word Support: https://support.microsoft.com/en-us/word
- Pandoc User Manual: https://pandoc.org/MANUAL.html
- Pandoc WASM (experimental): https://github.com/nicowillis/pandoc-wasm *(Note: Limited official documentation — see Appendix C)*

---

2. PROJECT DESCRIPTION

2.1 Reference Algorithm / Technologies

The platform builds upon and interfaces with the following established technologies:

- **Pandoc:** The primary conversion engine used to translate between document formats. Pandoc serves as the backbone of the bidirectional parser. *Note: Pandoc's WASM build currently lacks comprehensive documentation, which is an active constraint (see Appendix C).*
- **Typst:** A modern typesetting system with fast compilation and scripting capabilities. Typst's AST and scripting model are leveraged for parsing and rendering `.typ` documents.
- **LaTeX / TeX:** The long-standing standard for academic and scientific typesetting, referenced for its precision in mathematical content and complex layout rendering.
- **Abstract Document Model (ADM):** A custom intermediate representation developed as part of this project. The ADM is a format-neutral, structured data model storing document content, hierarchy, styling, and metadata — serving as the canonical source from which all format-specific representations are derived and synchronized.

2.2 SWOT Analysis

	Helpful	Harmful
Internal	**Strengths:** Solves a clear and pervasive pain point for users who work across multiple document formats. Offers a "best of all worlds" value proposition by unifying the strengths of LaTeX, Typst, Markdown, and Word in one workflow. Offline-first design ensures reliability and privacy.	**Weaknesses:** High development complexity, particularly for achieving robust and lossless bidirectional synchronization. The current underlying conversion tool (Pandoc) is unpolished and not fully featured in WASM environments. Feature-incomplete formats (e.g., Markdown) may cause irreversible data loss during round-trip conversions.
External	**Opportunities:** Targets high-value niches in academia, technical writing, and publishing where multi-format workflows are the norm. Can evolve into a backbone for collaborative editing platforms. Strong potential for CLI tooling and IDE plugin ecosystems.	**Threats:** Established tools such as Overleaf, Obsidian, and Microsoft Office 365 are gradually adding limited cross-format compatibility features. User inertia and reluctance to adopt a new, unproven workflow may slow adoption. Open-source alternatives may replicate core functionality.

2.3 Project Features

The following are the major features of the platform, organized by development phase:

Phase 1 — Core Engine (Current): - **Document Parsing:** Parse `.tex`, `.typ`, `.md`, and `.docx` files into the Abstract Document Model. - **Document Conversion and Independent Rendering:** Convert parsed ADM output into any supported format. Each format is rendered independently from the ADM, not from another format, to minimize conversion conflicts. - **Filesystem Management:** Manage document projects as structured directories, with project manifests tracking format variants. - **Offline Support and Sync:** Full offline operation with a local sync mechanism. Changes to any format variant trigger re-synchronization of all other variants via the ADM. *(Current milestone.)*

Phase 2 — Version Control and Export: - **Version Control and Management:** Track document history at the ADM level, enabling rollback, diff viewing, and conflict resolution across formats. - **Multi-Format Export:** Export documents to PDF, HTML, DOCX, plain text, and other

formats from a single command or UI action. - **Enhanced Editor with DOCX Support:** A richer in-app editor with direct support for editing Word-compatible documents.

Phase 3 — Developer and Power User Tools: - **Language Server Implementation:** Integrate LSP-compatible language servers for LaTeX and Typst to provide autocomplete, error diagnostics, and live preview within the unified editor. - **Document Hierarchy:** Support for multi-file document projects with include/import resolution (e.g., LaTeX `\input{}`, Typst `#include`).

Phase 4 — Collaboration and Roles: - **User Roles and Permissions:** Define contributor roles (viewer, editor, admin) for document projects. - **Project Management:** Manage collections of documents as projects with metadata, milestones, and task assignment. - **Real-Time Collaboration:** Concurrent multi-user editing with operational transformation or CRDT-based conflict resolution. - **Templates:** A library of document templates for common academic, technical, and professional formats.

Phase 5 — Validation and Analysis: - **Academic Structure Validation:** Validate document structure against academic conventions (e.g., section ordering, citation completeness, abstract word count). - **Format Compatibility Analyzer:** Analyze a source document and report which features may be degraded or lost when converting to a target format, with suggested mitigations.

2.4 User Classes and Characteristics

User Class	Description	Technical Proficiency	Primary Use Case
Academic User	Researchers, graduate students, professors	Moderate to high (familiar with LaTeX or Markdown)	Writing papers, theses, and reports across multiple submission formats
Technical Writer	Documentation engineers, developer advocates	High (comfortable with CLI and markup)	Maintaining single-source documentation across HTML, PDF, and DOCX outputs
Casual Writer	Undergraduate students, non-technical users	Low to moderate (familiar with Word)	Simple document creation with occasional format export needs
Developer / Power User	Engineers integrating the tool into build pipelines	High (CLI-first, scripting-friendly)	Automating document generation and format conversion in CI/CD workflows
Editor / Reviewer	Collaborators reviewing and annotating documents	Low to moderate	Commenting, reviewing, and approving document content

2.5 Design and Implementation Constraints

- **Hardware Constraints:** The application must function on consumer-grade hardware with a minimum of 4 GB RAM. Initial load performance is a known issue (see Appendix C) and must be optimized to meet the 2-second response threshold defined in Section 4.1.
- **Offline-First Architecture:** The system must operate fully without a network connection. All core features — parsing, conversion, rendering, and sync — must be available offline.
- **Pandoc Dependency:** The current conversion pipeline depends on Pandoc. The WASM build of Pandoc lacks official documentation and has known limitations in browser environments. The native binary is the primary supported runtime for the current phase.
- **Language Support:** The platform must support UTF-8 encoded documents in all formats. Right-to-left (RTL) language support is a future consideration and is not required in the current phase.
- **Cross-Platform Compatibility:** The application must run on Windows 10+, macOS 12+, and major Linux distributions (Ubuntu 22.04+, Fedora 38+).

- **Security Considerations:** All document data is stored locally. No telemetry or user data is transmitted without explicit consent. File access is sandboxed to the user's designated project directories.
- **Programming Standards:** TypeScript is the primary language for the frontend and synchronization engine. Rust or Go may be used for performance-critical parsing or CLI components. Code must conform to project-defined linting and formatting standards (ESLint, Prettier, Clippy).
- **Dependency Constraints:** Third-party runtime dependencies must be audited for license compatibility (MIT, Apache 2.0, or BSD preferred). GPL-licensed dependencies require explicit review before inclusion.

2.6 Design Diagrams

The following diagrams are to be developed and attached as part of Appendix B. Placeholders are listed below.

- **Use Case Diagram (Level 2):** Illustrating interactions between user classes and system features.
- **Class Diagram:** Defining the Abstract Document Model structure, format parsers, and synchronization engine components.
- **Activity Diagram:** Depicting the document edit → parse → sync → render workflow.
- **Sequence Diagram:** Showing the sequence of operations during a format conversion request.
- **State Diagram:** Representing the lifecycle states of a document (untracked, parsed, synced, conflicted, exported).

2.7 Assumptions and Dependencies

Assumptions: - Users will maintain one active project directory per document, and will not manually edit multiple format variants simultaneously outside the application. - Pandoc's native binary will remain available and maintain backward-compatible conversion output for supported formats. - Documents will primarily be authored in one canonical format per session, with others treated as derived outputs. - Feature parity between all formats is not expected or required; degradation of format-specific features (e.g., Typst scripting functions) during conversion is acceptable provided the information loss is disclosed and logged.

Dependencies:

- **Pandoc (native binary):** Required for

format conversion. Version ≥ 3.0.

- **Typst CLI:**

Required for compiling and rendering `.typ` documents.

- **Django** Required for the backend
- **LaTeX distribution (TeX Live / MiKTeX):** required only if users wish to compile `.tex` documents to PDF locally.

- **Operating system filesystem APIs:** Used for file watching,

project directory management, and offline storage.

---

3. SYSTEM REQUIREMENTS

3.1 User Interface

The user interface must provide the following views and functional areas:

- **Main Landing Page:**
  - Project browser showing recent and pinned document projects.
  - Quick-start options to create a new document or import an existing file.
  - Format selector to designate the canonical source format for a new project.
  - Application settings access.
- **Editing Pages:**
  - A unified editor with syntax highlighting for all supported formats (LaTeX, Typst, Markdown, DOCX preview).
  - A split-pane view enabling simultaneous display of two format representations.
  - A live preview pane rendering the document output (PDF, HTML, or rich text).
  - Format switcher to change the active editing format without leaving the document.
  - Inline conversion conflict indicators, highlighting areas where round-trip conversion resulted in data loss or substitution.
- **User Pages:**
  - User profile and preferences (editor theme, default format, key bindings).
  - Project settings (format variants enabled, version control configuration, export settings).
  - Notification center for sync events, conflicts, and export completions.
- **Offline Support and Sync View:**
  - Sync status dashboard showing the last successful sync time per format variant.
  - Conflict resolution interface for manually resolving divergent edits between format variants.
  - Format buffer log showing substitutions made during conversion (e.g., `#lorem(50)` $\longrightarrow$ expanded text $\longrightarrow$ retained as expanded text).

3.2 Software Interface

The platform consists of the following interconnected modules:

- **Editor Module:** The user-facing interface, built as a web or desktop application (Electron or Tauri). Communicates with the Synchronization Engine via an internal IPC or REST-like API. Sends document change events and receives updated ADM snapshots and conflict reports.

- **Abstract Document Model (ADM) Core:** The central data structure module. Receives parsed output from format-specific parsers, maintains the canonical document state, and emits format-specific ASTs to renderers. Implemented as a TypeScript library with a stable internal API.

- **Format Parsers:** Individual parser modules for each supported format (LaTeX, Typst, Markdown, DOCX). Each parser transforms a raw source file into an ADM-compatible intermediate representation. Parsers expose a standard `parse(filePath): ADMDocument` interface.
- **Synchronization Engine:** Orchestrates bidirectional sync between format variants. Consumes ADM diffs produced by parsers and applies them to all registered format renderers. Exposes a `sync(sourceFormat, targetFormats): SyncResult` API. Handles conflict detection and delegates resolution to the conflict manager.
- **Format Renderers:** Output modules that transform an ADM document back into a specific format file. Each renderer exposes a `render(adm: ADMDocument, targetFormat): File` interface.
- **Conversion Conflict Manager:** Intercepts conversion operations where the target format cannot faithfully represent a source feature (e.g., Typst scripting functions). Logs substitutions, stores original representations in a format buffer, and exposes conflict records to the UI.
- **Filesystem Manager:** Manages the project directory structure, file watchers, and project manifests. Monitors source files for external changes and triggers re-sync automatically.
- **CLI Interface:** A command-line wrapper exposing core operations (`sync`, `export`, `convert`, `status`) for integration into build pipelines and scripting environments.

3.3 Database Interface

The platform uses a local-first embedded database for storing project state, version history, and sync metadata. The following database approach is employed:

- **Primary Store:** A structured local store (e.g., SQLite via `better-sqlite3` or a document store such as LevelDB) is used to persist the ADM snapshots, version history, format buffer entries, and project metadata.
- **File-Based State:** Document content itself is stored as files on the local filesystem, not in the database. The database stores only metadata, diff logs, and format-specific auxiliary data.
- **Schema:** Key entities include `Project`, `DocumentVersion`, `FormatVariant`, `SyncEvent`, `ConversionBuffer`, and `ConflictRecord`.
- **No Remote Database:** In the current phase, no remote database or cloud sync service is used. All data remains local. Future phases may introduce optional sync with a user-configured remote endpoint.

3.4 Protocols

- **Inter-Process Communication (IPC):** The editor frontend communicates with the synchronization engine backend via a local IPC channel (Electron IPC or Tauri commands). All messages are JSON-serialized.
- **File Watching Protocol:** The filesystem manager uses OS-native file system event APIs (e.g., `inotify` on Linux, `FSEvents` on macOS, `ReadDirectoryChangesW` on Windows) to detect external file modifications and trigger re-sync.

- **Sync Protocol:** Synchronization operations are event-driven. A change in any monitored format variant triggers a parse → ADM diff → render cycle. Sync operations are serialized per document to prevent race conditions.
- **Export Protocol:** Export requests are queued and processed asynchronously. Progress events are emitted to the UI via the IPC channel. Export to PDF invokes Typst or LaTeX CLI tools as subprocesses, with stdout/stderr captured for error reporting.
- **Security:** All operations are local. No data leaves the user's machine without explicit export or user-configured sync. Subprocess invocations (Pandoc, Typst, LaTeX) are sandboxed to the project directory using restricted file-system access where the platform supports it.
- **Encryption:** Document files are stored as plaintext on the local filesystem. Users who require encryption are advised to use OS-level filesystem encryption (e.g., FileVault, BitLocker, LUKS). Application-level encryption is a future consideration.

---

4. NON-FUNCTIONAL REQUIREMENTS

4.1 Performance Requirements

- **Edit Responsiveness:** The system must reflect document edits in the active editor view within 100 milliseconds. Syntax highlighting and in-editor feedback must not lag behind user input.
- **Sync Throughput:** A full synchronization cycle (parse → ADM diff → render for all active format variants) must complete within **2 seconds** for documents up to 50,000 words under normal operating conditions.
- **Export Speed:** Document export to PDF must complete within 10 seconds for standard-length documents (up to 100 pages) when using Typst as the rendering backend. LaTeX export times are acknowledged to be longer due to compilation overhead.
- **Initial Load Performance:** Application startup and initial project load must complete within 5 seconds on the minimum supported hardware configuration. *(Known issue — see Appendix C.)*
- **Memory Usage:** The application must not consume more than 512 MB of RAM during normal single-document editing sessions.

4.2 Security Requirements

- **Local Data Sovereignty:** All user document data, project metadata, and version history must be stored exclusively on the user's local machine. No data is transmitted to external servers by default.
- **Process Isolation:** External tool invocations (Pandoc, Typst, LaTeX CLI) must be executed as isolated subprocesses with access restricted to the project directory. Shell injection via document content must be prevented by passing arguments as arrays, never as interpolated shell strings.
- **Dependency Auditing:** All third-party dependencies must be audited for known vulnerabilities prior to release. A dependency lock file must be maintained and checked into version control.

- **User Privacy:** No usage analytics, crash telemetry, or document content is collected or transmitted without explicit opt-in from the user.
- **Access Control (Future):** When user roles and permissions are introduced (Phase 4), access control enforcement must occur at the server layer, not only the client layer.

---

5. OTHER REQUIREMENTS

- **Accessibility:** The user interface must meet WCAG 2.1 Level AA accessibility standards, including keyboard navigability, sufficient color contrast, and screen reader compatibility for all primary workflows.
- **Localization:** The application interface must support internationalization (i18n) from the initial release, with English as the default locale. RTL layout support is deferred to a future release.
- **Licensing:** The platform must be releasable under an open-source license (MIT or Apache 2.0 preferred). Any GPL-licensed dependency must be isolated and clearly documented.
- **Extensibility:** The parser and renderer architecture must be designed as a plugin system, allowing third-party contributors to add support for additional formats without modifying the core ADM library.
- **Testing:** The synchronization engine must have unit test coverage of at least 80%, with integration tests covering all supported format conversion round-trips.
- **Documentation:** All public-facing APIs (ADM Core, Format Parsers, Sync Engine) must be documented with JSDoc or equivalent, and a developer guide must be maintained alongside the codebase.

Appendix A: Glossary

Term	Definition
ADM	Abstract Document Model. The format-neutral, intermediate representation of a document used as the canonical source of truth within the synchronization engine.
AST	Abstract Syntax Tree. A tree representation of the syntactic structure of a source document, produced by a format-specific parser.
Bidirectional Parser	A component capable of both reading a format into the ADM and writing the ADM back into that format.
Canonical Format	The user-designated primary format for a document, from which the initial ADM is derived.

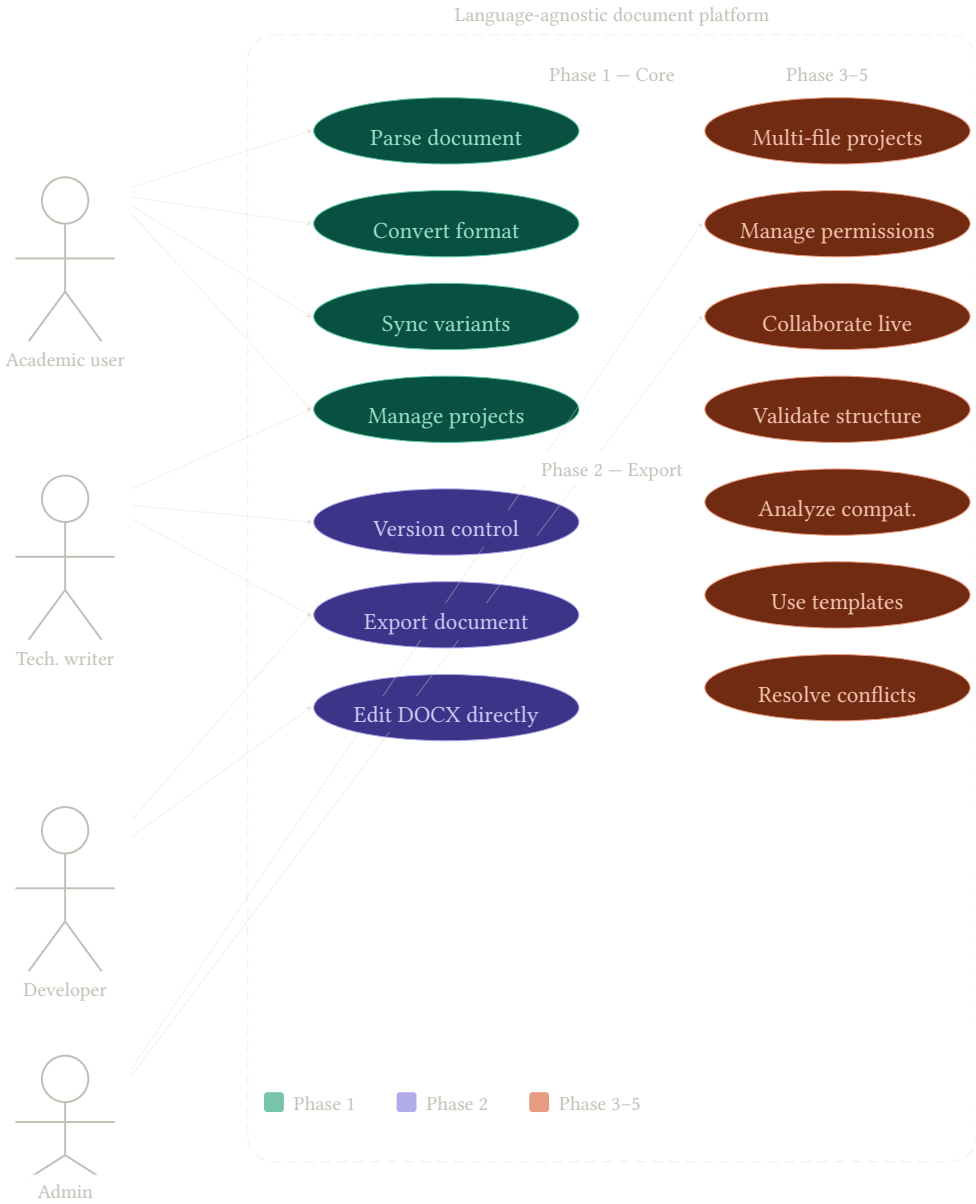
Term	Definition
CLI	Command-Line Interface. A text-based interface for interacting with the platform via terminal commands.
Conflict	A state in which two or more format variants of the same document have diverged in a way that cannot be automatically resolved by the synchronization engine.
Conversion Buffer	A log maintained by the Conflict Manager that records format-specific constructs (e.g., Typst functions) that were substituted or expanded during conversion, enabling partial reversibility.
CRDT	Conflict-free Replicated Data Type. A data structure designed for use in distributed systems to enable automatic conflict resolution in concurrent edits (planned for Phase 4).
DFD	Data Flow Diagram. A diagram representing how data moves between components of the system.
Format Variant	One of the supported format representations (`.tex`, `.typ`, `.md`, `.docx`) of a given document within a project.
IPC	Inter-Process Communication. The mechanism by which the editor frontend communicates with the backend synchronization engine.
LaTeX	A document preparation system based on the TeX typesetting engine, widely used in academia and scientific publishing.
LSP	Language Server Protocol. A protocol enabling editor-agnostic language support features (autocomplete, diagnostics) via a language server process.
Markdown	A lightweight plaintext markup language designed for simple formatting and easy readability.

Term	Definition
Pandoc	An open-source universal document converter used as the primary conversion engine in this platform.
Round-Trip Conversion	The process of converting a document from Format A → ADM → Format B → ADM → Format A, and assessing the fidelity of the final result relative to the original.
SRS	Software Requirements Specification. This document.
Sync Cycle	One complete execution of the parse → ADM diff → render pipeline triggered by a detected change in a format variant.
Typst	A modern, compiled typesetting language with fast compilation and scripting support, used as an alternative to LaTeX.
WASM	WebAssembly. A binary instruction format enabling execution of compiled code in web browser environments.

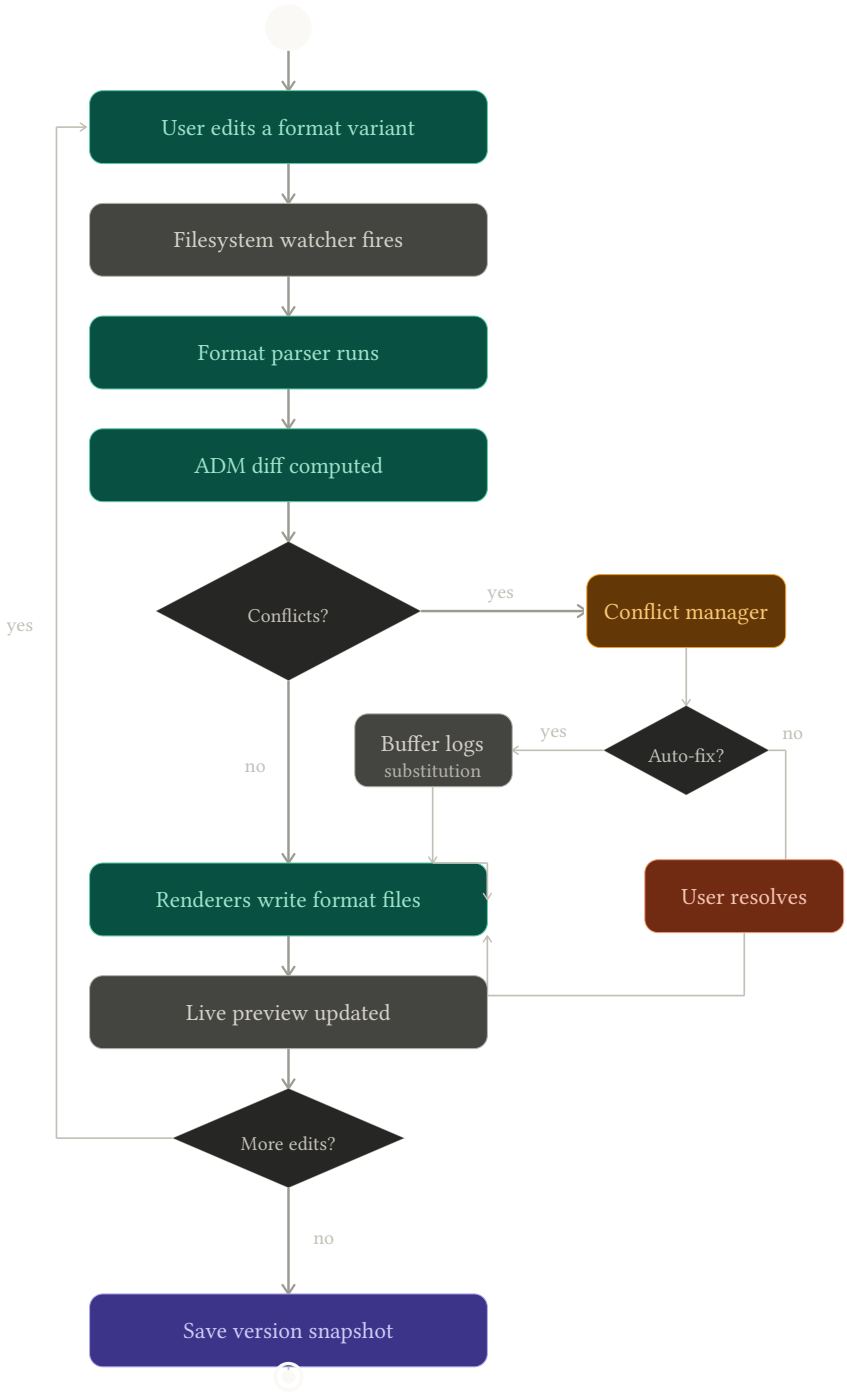
Appendix B: Analysis Models

The following analysis models are to be finalized and attached to this SRS. All diagrams are under development.

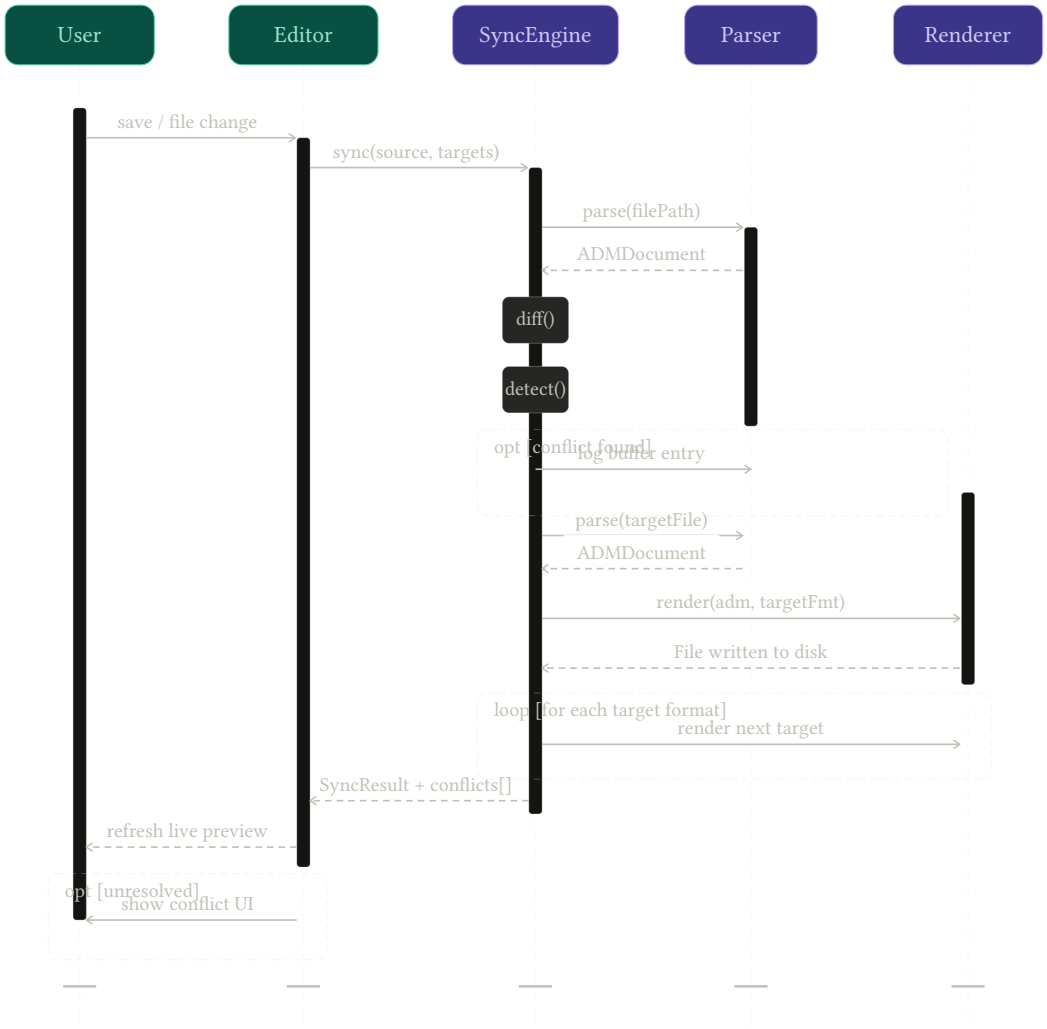
- **B.1 — Use Case Diagram (Level 2):** Full feature-level use cases for all user classes.



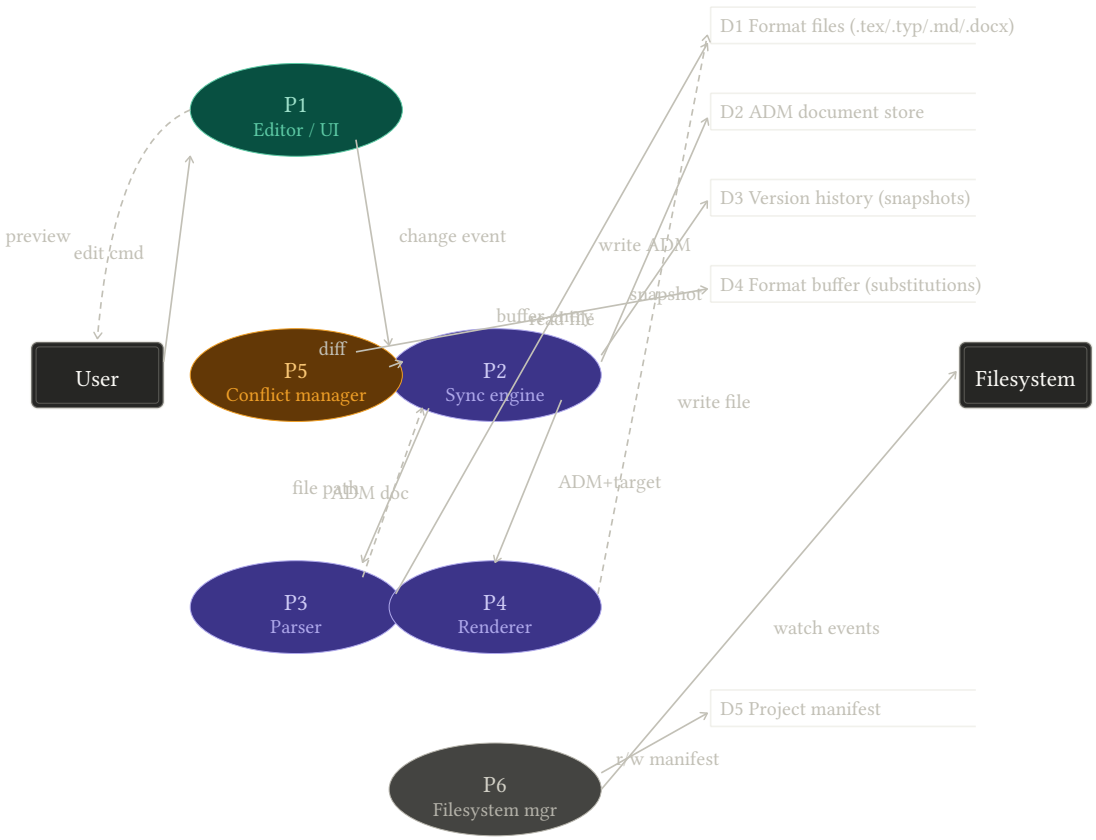
- **B.2 — Class Diagram:** ADM structure, parser interfaces, renderer interfaces, and sync engine components.



- **B.3 — Activity Diagram:** Document edit and synchronization workflow.



- **B.4 — Data Flow Diagram:** Data movement across editor, ADM, filesystem, and renderers.



Appendix C: Issues List

This is a dynamic list of open requirements and known technical issues. Items are tracked and updated as the project progresses.

ID	Issue	Status	Priority	Notes
ISS-001	**No official documentation for Pandoc's WASM build.** Pandoc's WebAssembly port lacks comprehensive documentation, making browser-based conversion unreliable.	Open	High	Currently using Pandoc native binary as the primary runtime. WASM integration deferred until documentation matures or an alternative WASM converter is identified.
ISS-002	**Performance degradation on initial application load.**	Open	High	Profiling required to identify the bottleneck. Suspected

ID	Issue	Status	Priority	Notes
	The application exhibits slow startup times, particularly when loading large projects or initializing the Pandoc runtime. Surprisingly, high-volume text editing does not reproduce this issue.			causes include Pandoc subprocess cold-start latency and synchronous filesystem scanning during project initialization.
ISS-003	**Conversion conflicts when round-tripping through feature-incomplete formats.** Converting a Typst source document to Markdown and back to Typst results in the loss of Typst-specific scripting constructs. Example: `#lorem(50)` is correctly expanded to 50 words of Lorem Ipsum text in Markdown, but upon conversion back to Typst, the original function call is replaced with the expanded literal text, destroying the original semantic intent.	Open	High	A format-specific **Conversion Buffer** mechanism is proposed: the sync engine stores a mapping of original constructs to their expanded equivalents and attempts to restore them during reverse conversion. Design for this buffer is in progress.
ISS-004	**Conflict resolution strategy for simultaneous edits across format variants.** When a user edits two format variants of the same document concurrently (e.g., editing `.md` in an external editor while the app has `.tex` open), the ADM may receive conflicting diff inputs.	Open	Medium	A last-write-wins strategy is the current default. A structured conflict resolution UI (as specified in Section 3.1) is planned to allow manual resolution.
ISS-005	**LaTeX custom command handling in the bidirectional parser.** Documents using custom LaTeX macros (`\newcommand`, `\def`) are not	Open	Medium	Partial workaround: user-defined macro tables can be supplied as a project-level configuration file. Full resolution re-

ID	Issue	Status	Priority	Notes
	reliably parsed into the ADM, as the parser cannot resolve macro definitions without executing the TeX engine.			quires integrating a TeX interpreter or pre-processing step.